American Computer Science League

2021-2022 ● Contest 4: Fibonacci & Mandelbrot ● Senior Division

PROBLEM: Given the function $f(z) = z^2 + C$, "iterate" the function by evaluating $f(f(f(\ldots f(f(0)))))$, stopping as soon as the function *escapes* or *cycles*. The function *escapes* when its absolute value is greater than a specified value; in this problem it is 4. The function *cycles* when it results in a value that it has already produced after rounding each result to 3 decimal places. The variable z and the constant C are *complex* numbers. You'll be evaluating a set of values of C and reporting information about the evaluation of f.

In order to use *complex* numbers, we need to define an *imaginary* number, *i*, such that $i = \sqrt{-1}$, and therefore, the value of $i^2 = \sqrt{-1} * \sqrt{-1} = -1$. A *complex* number has two parts: a *real* part and an *imaginary* part. They are represented in the form $a + bi$, where a is referred to as the *real* number part and bi as the *imaginary* number part. Both *a* and *b* are *real* numbers.

There are three operations you will need to perform on *complex* numbers: addition, multiplication, and absolute value. Here is how each operation is performed:

- **Addition.** Combine the *real* number parts and combine the *imaginary* number parts:

  $$(a + bi) + (c + di) = (a + c) + (b + d)i$$

  Example: $(5 + 2i) + (3 - 6i) = 8 - 4i$

- **Multiplication.** Use the distributive property on binomial terms:

  $$(a + bi)(c + di) = ac + adi + bci + bdi^2$$
  $$= (ac - bd) + (ad + bc)i$$

  Example: $(5 + i)(3 + 2i) = 5 \cdot 3 + 5 \cdot 2i + 1 \cdot 3i + 1 \cdot 2i^2$
  $$= 15 + 10i + 3i - 2$$
  $$= 13 + 13i$$

  Example: $(5 + 2i)(3 - 6i) = 27 - 24i$

- **Absolute Value.** The absolute value of a *complex* number is given by the formula:

  $$|a + bi| = \sqrt{a^2 + b^2}$$

  Example: $|3 - 4i| = \sqrt{3^2 + (-4)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$.

Here are two examples that show how to evaluate f. In the first example, the function *escapes* because the evaluation stops if the absolute value of the function is greater than 4. In the second example, the function *cycles* and evaluation stops because the value of the function, always rounded to three decimal places, has already appeared. When rounding, find the closest value to the calculated result with only 3 decimal places and if there's a 5 in the 4th decimal place, round

away from 0. This is the default for the *round* function in Java, C++, and Python. We guarantee that there will be no more than 500 iterations before the function either *cycles* or *escapes*.

EXAMPLE #1: $C = 2 - 0.3i$

$f(0) = (0)^2 + (2 - 0.3i) = 2 - 0.3i$ (This absolute value < 4.)

$f(f(0)) = f(2 - 0.3i) = (2 - 0.3i)^2 + (2 - 0.3i)$

$= (4 - 0.6i - 0.6i + 0.09i^2) + (2 - 0.3i)$

$= (4 - 0.09 + 2) + (-0.6 - 0.6 - 0.3)i$

$= 5.91 - 1.5i$

The absolute value of $5.91 - 1.5i = \sqrt{5.91^2 + (-1.5)^2} = \sqrt{34.9281 + 2.25} = 6.09738$. This value is larger than 4, so the evaluation stops.

EXAMPLE #2: $C = -0.065 - 0.2i$

$f(0) = (0)^2 + (-0.065 - 0.2i) = -0.065 - 0.2i$

$f(f(0)) = f(-0.065 - 0.2i) = (-0.065 - 0.2i)^2 + (-0.065 - 0.2i) = -0.101 - 0.174i$

$f(f(f(0))) = f(-0.101 - 0.174i) = (-0.101 - 0.174i)^2 + (-0.065 - 0.2i) = -0.085 - 0.165i$

$f(f(f(f(0)))) = f(-0.085 - 0.165i) = (-0.085 - 0.165i)^2 + (-0.065 - 0.2i) = -0.085 - 0.172i$

$f(f(f(f(f(0))))) = f(-0.085 - 0.172i) = (-0.085 - 0.172i)^2 + (-0.065 - 0.2i) = -0.087 - 0.171i$

$f(f(f(f(f(f(0)))))) = f(-0.087 - 0.171i) = (-0.087 - 0.171i)^2 + (-0.065 - 0.2i) = -0.087 - 0.17i$

$f(f(f(f(f(f(0)))))) = f(-0.087 - 0.17i) = (-0.087 - 0.17i)^2 + (-0.065 - 0.2i) = -0.086 - 0.17i$

$f(f(f(f(f(f(0)))))) = f(-0.086 - 0.17i) = (-0.086 - 0.17i)^2 + (-0.065 - 0.2i) = -0.087 - 0.171i$

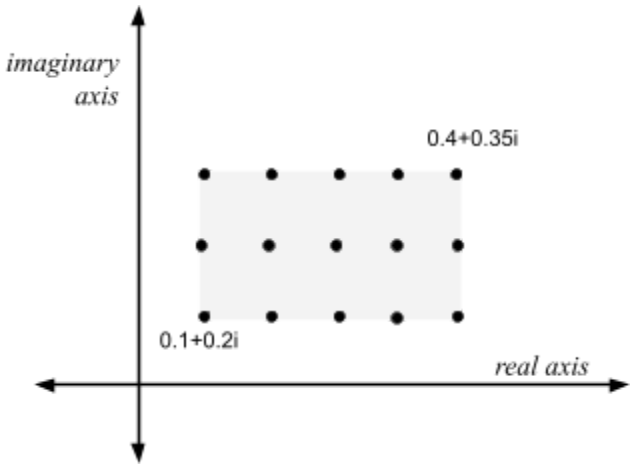
Since the absolute value never exceeds 4 and this last answer is the same value as the 5th result, the evaluation stops. The length of the resulting *cycle* is 3; that is, the function would *cycle* through 3 different values even though other values preceded them.

In this program, you will be given 2 complex numbers and an increment amount. These numbers define a rectangular area in the *complex* plane and points of interest in that area. The *complex* number $a + bi$ is plotted in the *complex* plane as *(a, b)* with *a* along the *real* axis and *b* along the *imaginary* axis. For example, "0.1 0.2 0.4 0.35 0.075" indicates that the points (0.1, 0.2) and (0.4, 0.35) are the lower-left and the upper-right corners of the rectangular area respectively and the increment amount is 0.075. Every value being checked should initially be rounded to 3 decimal places (both the *real* part and the *imaginary* part) to avoid roundoff error with floating point numbers. We guarantee that the increment will be smaller than either range of values and will include the points on the perimeter in every case. In this example, the increment

amount added to both *a* and *b* from the lower-left corner to upper-right corner of the rectangular area will generate the following 15 *complex* numbers:

(0.1+0.35*i*)	(0.175+0.35*i*)	(0.25+0.35*i*)	(0.325+0.35*i*)	(0.4+0.35*i*)
(0.1+0.275*i*)	(0.175+0.275*i*)	(0.25+0.275*i*)	(0.325+0.275*i*)	(0.4+0.275*i*)
(0.1+0.2*i*)	(0.175+0.2*i*)	(0.25+0.2*i*)	(0.325+0.2*i*)	(0.4+0.2*i*)

Graphically, the 15 *complex* numbers are plotted as follows:



For each *complex* number that is generated, iterate the function until a *cycle* is found or the function *escapes*. At each step, round the result to the nearest one-thousandth (both the *real* part and the *imaginary* part). For functions that do not result in an *escape*, a *cycle* will eventually emerge. Record the length of the *cycle*. Output the number of generated *complex* numbers having a *cycle* length that is a Fibonacci number (1, 2, 3, 5, 8, 13, 21, 34, ….) less than 500.

EXAMPLE #3: Input 0.1 0.2 0.4 0.35 0.075

The following values of *C* are evaluated. The bold number to the right is 0 if the function *escapes*, or a positive number indicating the length of the *cycle*.

(0.1+0.2*i*) **1**	(0.175+0.2*i*) **1**	(0.25+0.2*i*) **1**	(0.325+0.2*i*) **6**	(0.4+0.2*i*) **0**
(0.1+0.275*i*) **1**	(0.175+0.275*i*) **1**	(0.25+0.275*i*) **1**	(0.325+0.275*i*) **6**	(0.4+0.275*i*) **0**
(0.1+0.35*i*) **1**	(0.175+0.35*i*) **1**	(0.25+0.35*i*) **4**	(0.325+.035*i*) **5**	(0.4+0.35*i*) **105**

There are 2 values of *C* that *escape*. Of the 13 other *C* values, 9 of them have *cycles* whose lengths are Fibonacci numbers.

INPUT: Each data set has 5 decimal values representing the lower-left corner and the upper-right corner respectively of a rectangular region on the *complex* plane. Both *complex*

numbers represent a point *(a,b)* for a *complex* number *a +bi*. The 5th number is the increment between the points on both the *real* axis and the *imaginary* axis.

OUTPUT: For each line of data, find the total number of *C* values that have a *cycle* length that is equal to one of the numbers in the Fibonacci sequence.

SAMPLE INPUT:

```
0.1 0.2 0.4 0.35 0.075
-0.1 -0.1 0.0 0.1 0.05
-2.5 -1.0 -2.0 0.0 0.05
-0.1 -0.3 -0.05 -0.2 0.005
-0.3 -0.3 0.2 0.2 0.1
```

SAMPLE OUTPUT:

```
1. 9
2. 15
3. 1
4. 222
5. 34
```

American Computer Science League

2021-2022 ● Contest 4: Fibonacci & Mandelbrot ● Senior Division

TEST DATA

TEST INPUT:

```
-1.5 -1.5 1.5 1.5 0.2
-0.4 -0.3 0.4 0.3 0.06
0.175 -0.1 0.235 0.4 0.006
-0.2 -0.2 0.0 0.5 0.02
-0.375 -0.1 0.075 0.5 0.025
```

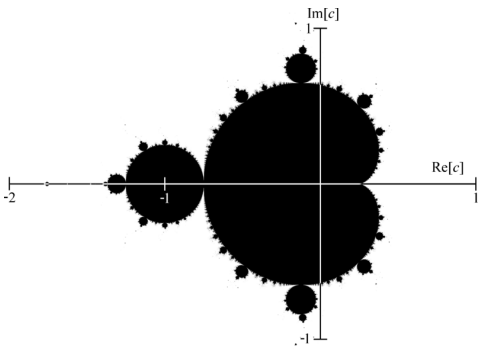
TEST OUTPUT:

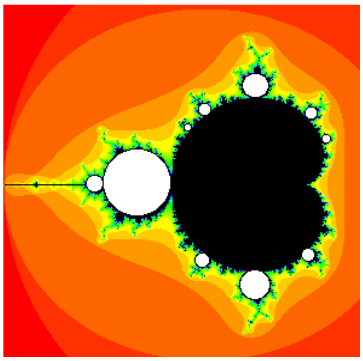
1. 30
2. 127
3. 764
4. 388
5. 456

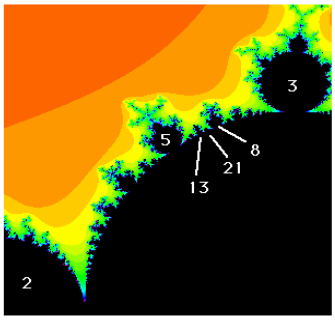
American Computer Science League

2021-2022 ● Contest 4: Fibonacci & Mandelbrot ● Senior Division

BACKGROUND INFORMATION FOR THE ADVISOR:







The Mandelbrot Set is generated by iterating the function $f(z) = z^2 + C$ for points within a small range on the complex plane where the point (a,b) represents the complex number a + bi as shown in the 1st illustration.

When iterating a function over itself, the function either converges to a single value, cycles between multiple values, or "escapes" to infinity, meaning that its value gets larger each time. If you are given an initial value for C, a complex number, starting with a value of z = 0, the function generates a sequence of numbers until one of these occurs.

If the iteration results in converging or cycling among 2 or more values, the point is colored back because it is in the Mandelbrot Set. The various colors outside the Mandelbrot Set are used to illustrate how many iterations it takes to exceed some value, thus how quickly the values "escape" to infinity.

The main "bulb" (black area in the 2nd illustration) in the Mandelbrot Set represents all points that converge or have a cycle of length 1. The smaller bulbs (white areas in the 2nd illustration) represent points with different cycle lengths.

The Fibonacci sequence can be found in the Mandelbrot Set as the length of the cycle for all of the points in each of the bulbs in the upper left and lower left sections of the main bulb as shown in the 3rd illustration given.

American Computer Science League

2021-2022 ● Contest 4: Fibonacci & Mandelbrot ● Senior Division

问题: 给定一个函数 $f(z) = z^2 + C$，通过反复迭代函数 $f(z)$，计算 $f(f(f(\ldots f(f(0)))))$ 的值。当**函数** $f(z)$ 遇到“ escapes ”或者“ cycles ”现象时，计算便会**停止。当函数的绝对值大于指定的值，**表明**函数** $f(z)$ 遇到“ escapes ”现象**（这**个题目中**，指定**的**值是** 4**）。当函数** $f(z)$ 产生一个值，且生成的每个结果都四舍五**入到小数点后** 3 **位，**表明**函数** $f(z)$ 遇到了“ cycles ”现象。**变量** z **和常数** C **是复数。你将**要**计算一**组 C 的**值，并且**报告函数 f 的评估**信息。**

为了使用复数，我们需要定义一个虚数, i , 为此，我们定义 $i = \sqrt{-1}$, 于是可以得到 $i^2 = \sqrt{-1} * \sqrt{-1} = -1$。一个复数有两个部分：实部和虚部，以 $a + bi$ 的形式表示，其中 a 表示实部，bi 表示虚部。a 和 b 均为实数。

你**需要对复数**执行三种运算：加法、乘法和取绝对值。以下是每种运算的执行方式：

- **加法.**实部和实部相加，虚部和虚部相加：

  $$(a + bi) + (c + di) = (a + c) + (b + d)i$$

  例如: $(5 + 2i) + (3 - 6i) = 8 - 4i$

- **乘法. 使用二项**式的**乘法分配律**：

  $$(a + bi)(c + di) = ac + adi + bci + bdi^2$$
  $$= (ac - bd) + (ad + bc)i$$

  例如: $(5 + i)(3 + 2i) = 5 \cdot 3 + 5 \cdot 2i + 1 \cdot 3i + 1 \cdot 2i^2$
  $$= 15 + 10i + 3i - 2$$
  $$= 13 + 13i$$

  例如: $(5 + 2i)(3 - 6i) = 27 - 24i$

- **绝对值.**根据给定的下述**公式**计算**复数的绝对值** ：

  $$|a + bi| = \sqrt{a^2 + b^2}$$

  例如: $|3 - 4i| = \sqrt{3^2 + (-4)^2} = \sqrt{9 + 16} = \sqrt{25} = 5.$

这里**有两个例子展示如何计算** f。在第一个示例中，由于函数的绝对值大于 4，停止计算，致使函数将遇到“ escapes ”现象。在第二个示例中，由于已经出现的函数值总是四舍五入到小数点后三位，致使函数遇到“ cycles ”现象，停止计算。四舍五入时，找到与小数点后仅三位的计算结果最接近的值，如果小数点后的第 4 位**是** 5，则从 0 开始四舍五入。

American Computer Science League

2021-2022 ● Contest 4: Fibonacci & Mandelbrot ● Senior Division

这是 Java、C++ 和 Python **中四舍五入函数的默认**方式。**我们保证**在函数遇到“ cycles ”现象或者“ escapes ”现象**前，**函数的迭代次数**不会超过** 500 **次。**

例子 #1: $C = 2 - 0.3i$

$f(0) = (0)^2 + (2 - 0.3i) = 2 - 0.3i$ (绝对值 < 4.)

$f(f(0)) = f(2 - 0.3i) = (2 - 0.3i)^2 + (2 - 0.3i)$

$= (4 - 0.6i - 0.6i + 0.09i^2) + (2 - 0.3i)$

$= (4 - 0.09 + 2) + (-0.6 - 0.6 - 0.3)i$

$= 5.91 - 1.5i$

绝对值 $5.91 - 1.5i = \sqrt{5.91^2 + (-1.5)^2} = \sqrt{34.9281 + 2.25} = 6.09738.$

该**绝对值大于** 4，所以**计算停止**。

例子 #2: $C = -0.065 - 0.2i$

$f(0) = (0)^2 + (-0.065 - 0.2i) = -0.065 - 0.2i$

$f(f(0)) = f(-0.065 - 0.2i) = (-0.065 - 0.2i)^2 + (-0.065 - 0.2i) = -0.101 - 0.174i$

$f(f(f(0))) = f(-0.101 - 0.174i) = (-0.101 - 0.174i)^2 + (-0.065 - 0.2i) = -0.085 - 0.165i$

$f(f(f(f(0)))) = f(-0.085 - 0.165i) = (-0.085 - 0.165i)^2 + (-0.065 - 0.2i) = -0.085 - 0.172i$

$f(f(f(f(f(0))))) = f(-0.085 - 0.172i) = (-0.085 - 0.172i)^2 + (-0.065 - 0.2i) = -0.087 - 0.171i$

$f(f(f(f(f(f(0)))))) = f(-0.087 - 0.171i) = (-0.087 - 0.171i)^2 + (-0.065 - 0.2i) = -0.087 - 0.17i$

$f(f(f(f(f(f(0)))))) = f(-0.087 - 0.17i) = (-0.087 - 0.17i)^2 + (-0.065 - 0.2i) = -0.086 - 0.17i$

$f(f(f(f(f(f(0)))))) = f(-0.086 - 0.17i) = (-0.086 - 0.17i)^2 + (-0.065 - 0.2i) = -0.087 - 0.171i$

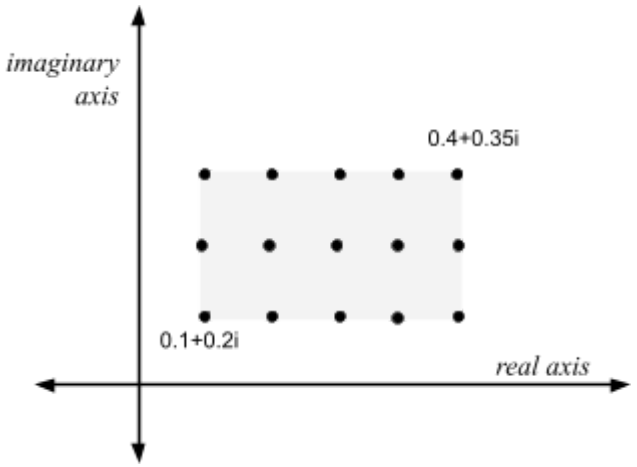
由于绝对值从不超过 4，且最后一个答案的值与第五个结果的值相同，因此**计算**将停止。由此函数产生的循环长度为 3；也就是说，函数将在 3 个不同的值之间循环，即使**在这**之前有其他值。

在这个程序中，你将得到 2 个复数和一个增量。这些数字定义了在复**数**平面中的矩形区域以及该区域中的**兴趣**点。复数 $a + bi$ 在**复数**平面中绘制为（a , b），其中 a 沿**实**轴，b 沿**虚**轴。例如，“0.1 0.2 0.4 0.35 0.075” 表示点（0.1，0.2）和（0.4, 0.35）分别为矩形区域的左下角和右上角，增量为 0.075。检查的每个值最初都应四舍五入到小数点后 3 位（**实部和虚部**），以避免浮点数的舍入错误。我们保证增量将小于任何一个数的值，并且在任何情况下都将包括矩形边上的点。

在本例中，从矩形区域的左下角到右上角添加到 a 和 b 的增量将生成以下 15 个复数：

(0.1+0.35*i*)	(0.175+0.35*i*)	(0.25+0.35*i*)	(0.325+0.35*i*)	(0.4+0.35*i*)
(0.1+0.275*i*)	(0.175+0.275*i*)	(0.25+0.275*i*)	(0.325+0.275*i*)	(0.4+0.275*i*)
(0.1+0.2*i*)	(0.175+0.2*i*)	(0.25+0.2*i*)	(0.325+0.2*i*)	(0.4+0.2*i*)

从图形上看，15 个复数绘制如下：



对于生成的每个复数，迭代函数，直到发现函数的“ cycles ”现象或者“ escapes ”现象。在每一步中，将结果四舍五入到**其**最接近的千分位（**实部和虚部**）。对于没有出现“ escapes ”现象**的函数**，其最终肯定会出现“ cycles ”现象。记录循环的长度。输出生成的复数的个数，且该复数的循环长度值是斐波那契数**列**中（1,2,3,5,8,13,21,34，…）**小于** 500 的数。

例子 #3: 输入 0.1 0.2 0.4 0.35 0.075

计算以下 C 的值。如果函数遇到“ escapes ”现象，则右边的粗体数字是 0，或者是一个表示循环长度的正数。

(0.1+0.2*i*) **1**	(0.175+0.2*i*) **1**	(0.25+0.2*i*) **1**	(0.325+0.2*i*) **6**	(0.4+0.2*i*) **0**
(0.1+0.275*i*) **1**	(0.175+0.275*i*) **1**	(0.25+0.275*i*) **1**	(0.325+0.275*i*) **6**	(0.4+0.275*i*) **0**
(0.1+0.35*i*) **1**	(0.175+0.35*i*) **1**	(0.25+0.35*i*) **4**	(0.325+.035*i*) **5**	(0.4+0.35*i*) **105**

有两个 C 值表示函数遇到了“ escapes ”现象。在其他 13 个 C 值中，有 9 个数是表示循环长度的正数，且属于斐波那契数**列**。

输入: 每个数据集有 5 个十进制数，分别代表复数平面上矩形区域的左下角和右上角。两个复数都代表复数 $a + b\,i$ 的一个点（ a , b ）。第五个数字是实轴和虚轴上点之间的增量。

输出：对于每行数据，找出循环长度**等于斐波那契数列中一个数字的** C **值的总**个**数。**

样本输入:

```
0.1 0.2 0.4 0.35 0.075
-0.1 -0.1 0.0 0.1 0.05
-2.5 -1.0 -2.0 0.0 0.05
-0.1 -0.3 -0.05 -0.2 0.005
-0.3 -0.3 0.2 0.2 0.1
```

样本输出:

1. 9
2. 15
3. 1
4. 222
5. 34

American Computer Science League

2021-2022 ● Contest 4: Fibonacci & Mandelbrot ● Senior Division

测试数据

测试输入:

```
-1.5 -1.5 1.5 1.5 0.2
-0.4 -0.3 0.4 0.3 0.06
0.175 -0.1 0.235 0.4 0.006
-0.2 -0.2 0.0 0.5 0.02
-0.375 -0.1 0.075 0.5 0.025
```

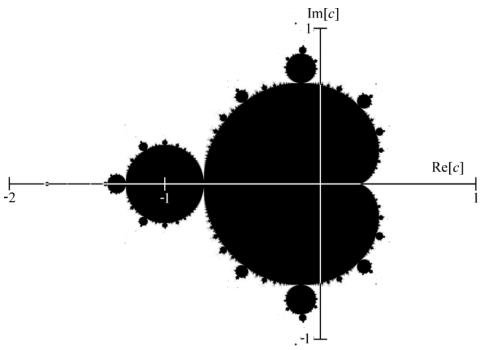
测试输出：

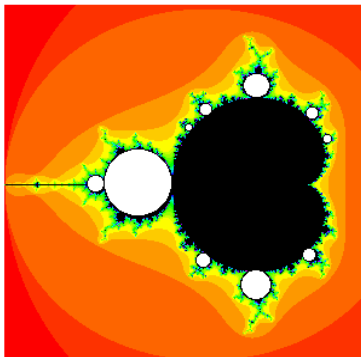
1. 30
2. 127
3. 764
4. 388
5. 456

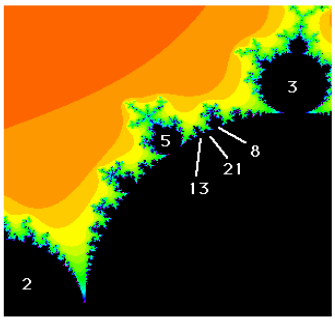
American Computer Science League

2021-2022 ● Contest 4: Fibonacci & Mandelbrot ● Senior Division

背景信息:







曼德布洛特**集合是通过迭代函数** $f(z) = z^2 + C$**，**在复**平面上**输出**小**范围内**的点**而生成的，**其中点（**a , b**）表示复数** $a + bi$**，如图 1 所示。**

当函数进行**自**我**迭代时，函数要么收敛**至**一个值，在多个值之间**“ 反复**循环 ”，要么**“ escapes ”到无穷大，这意味着它的值每次都会变大。如果给你一个复数** C **的初始值，**且**从** $z = 0$ **开始，**该**函数会生成一**系列**数字，直到**出现上述现象中的**一个。**

如果迭代的结果是在 2 个或多个值之间收敛或循环，则该点的背景会被着色，因为它位于曼德布洛特**集合中。曼德**布洛特**集合外的各种颜色是用**来**说明需要多少次迭代才能超过某个值，从而说明这个值“ escapes ”到无穷大的速度**有多快。

曼德布洛特**集合中的“ 主**灯泡区域 ”**（第二幅图中的黑色区域）表示所有收敛或循环长度为 1 的点。较小的“** 灯泡区域 ”**（第二幅图中的白色区域）代表具有不同循环长度的点。**

如给出的第 3 幅图所示，在“ 主灯泡区域 ”的左上部分和左下部分中，每个“ 灯泡区域 ”中表示循环长度的所有点都能够在曼德布洛特集合中找到斐波那契数列。